

Delegated Sessions

openSession already takes the session's owner as its first argument. The gateway passes its own address.

The problem

The hosted gateway multiplexes arbitrarily many Web2 callers onto sessions funded by one consumer wallet, behind careful engineering: idle-session claiming under `FOR UPDATE SKIP LOCKED`, a cheapest-bid walk with per-provider cool-off, spend fuses that ship inert and arm per environment. Better fuses do not reach what is left: one balance is a liveness dependency for every caller at once; per-caller accounting lives in `credits_ledger` and Redis, which the chain cannot correct, while `routed_sessions` carries no `user_id`; and no caller has a claim on the session serving them. The address that staked, that gets refunded, and that `getUserSessions` returns is yours.

Mechanism

The diamond carries a `Delegation` facet initialised against the `delegate.xyz v2` registry. `openSession(address user_, ...)` calls `_validateDelegatee` with `DELEGATION_RULES_SESSION` at `SessionRouter.sol:90`, then pulls stake from `user_` at `:104`: caller and owner are separate variables, and `SessionRouter.test.ts:208` already tests the split. Nothing on chain has to change for a gateway to stop holding its users' money. One argument in the Go node forecloses it: `session_router.go:61` and `:78` pass `opts.From`, fed by `OpenSessionByBidId` reading `s.GetMyAddress(ctx)` (`service.go:1260`).

So the gateway holds a delegation instead of custody. The consumer writes `delegateContract(gateway, diamond, DELEGATION_RULES_SESSION, true)` to the registry itself; the gateway opens with `user_` set to that consumer. Stake leaves the consumer's balance and every exit is addressed back to them, `_rewardUserAfterClose` to `session.user` and `withdrawUserStakes` to `user_`. That is the loss bound: pooled, a compromised gateway key yields the pool; delegated, the worst it does is burn a stipend claim and day-lock that consumer's principal for under a day, since the contract offers it no other address.

Two caps already hold in deployed code and revoke without the gateway's help: the consumer's ERC-20 allowance to the diamond, and `maxSessionDuration`. A boolean `checkDelegateForContract` conveys nothing else, which should be said plainly: no per-open size limit, no expiry, no model restriction. The registry holds the first already, since `delegateERC20(to, MOR, rights, amount)` publishes an upper bound readable through `checkDelegateForERC20`; expiry and model scope want a small facet, which is the second move.

The key problem, head on. A delegation needs a consumer key, which account-native users lack, and that cost does not disappear. But the gateway knows which callers have one: `wallet_links` stores addresses proved by SIWE, linked today to aggregate staking allocations, and that cohort moves with no new onboarding. For the rest the pool stays, much smaller, with a sponsored delegation from a smart account as the route off it.

Morpheus gets

- Delegated consumers stake their own MOR and draw their own stipend, so one wallet's stake-days stop rationing everyone's throughput.
- Who consumed what, on chain in `userSessions[user_]` rather than in a ledger the chain cannot correct, which is why `_daylock_from_close_receipt` had to be written.

Hanzo and Lux get

- The shape `api.hanzo.ai` needs, which is why we will write it either way.

First step, and who takes it

Thread an optional session owner through `OpenSessionByBidId` into `session_router.go`, defaulting to `opts.From` so existing deployments behave identically. That is the whole node change, its contract tests exist, and Hanzo opens it against `dev`; the gateway then opens delegated for its SIWE-linked users first, behind the fuses it ships. Neither needs a facet upgrade or a 5-of-9 signature. Reading the ERC-20 cap in `_validateDelegatee` is the one contract change, small enough for a queued diamond cut.

What this does not solve. Delegation fixes who owns a session, not what the session bought. A delegated consumer is defrauded by a provider substituting its backend exactly as a pooled one is, which is LP-313.

LP-320, full paper available. Market is the diamond at `0x6aBE1d282f72B474E54527D93b979A4f64d3030a` on Base.